

Assignment-8.4

2303A51242

Batch-05

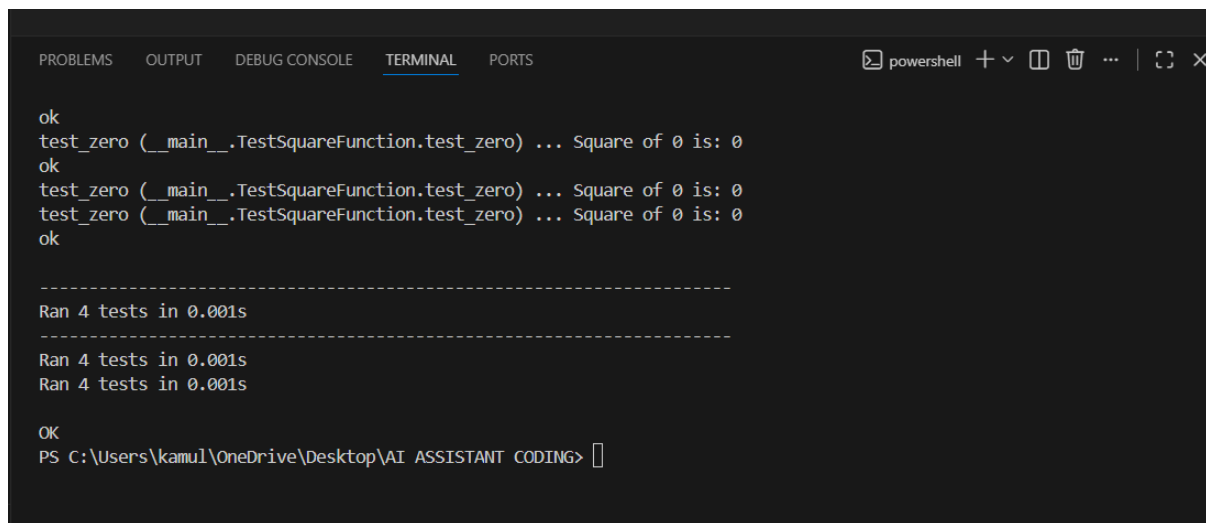
Task 1: Developing a Utility Function Using TDD

Prompt: Write Python unit tests using unittest for a square(x) function following TDD. After tests fail, implement square(x) in a separate file so all tests pass.

```
1.py | assignment-5.py | assignment-6.py | assignment-7.py | square_test.py | square.py
square_test.py > TestSquareFunction > test_positive_number
1  '''Write Python unit tests using unittest for a square(x) function following TDD.
2  After tests fail, implement square(x) in a separate file so all tests pass.
3  '''
4  import unittest
5  from square_test import square
6  class TestSquareFunction(unittest.TestCase):
7      def test_positive_number(self):
8          result = square(4)
9          print("Square of 4 is:", result)
10         self.assertEqual(result, 16)
11     def test_zero(self):
12         result = square(0)
13         print("Square of 0 is:", result)
14         self.assertEqual(result, 0)
15     def test_negative_number(self):
16         result = square(-5)
17         print("Square of -5 is:", result)
18         self.assertEqual(result, 25)
19     def test_decimal_number(self):
20         result = square(2.5)
21         print("Square of 2.5 is:", result)
22         self.assertEqual(result, 6.25)
23 if __name__ == "__main__":
24     unittest.main(verbosity=2)
```

```
palindrome_test.py > square
1  def square(x):
2      return x * x
```

Output:

A screenshot of a PowerShell terminal window. The window has a dark background and a title bar that says "powershell". The terminal output shows three "ok" messages, each followed by a test function call: "test_zero (__main__.TestSquareFunction.test_zero) ... Square of 0 is: 0". Below these, there are three lines of "Ran 4 tests in 0.001s" separated by dashed lines. At the bottom, it says "OK" and shows the PowerShell prompt "PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>".

```
ok
test_zero (__main__.TestSquareFunction.test_zero) ... Square of 0 is: 0
ok
test_zero (__main__.TestSquareFunction.test_zero) ... Square of 0 is: 0
test_zero (__main__.TestSquareFunction.test_zero) ... Square of 0 is: 0
ok

-----
Ran 4 tests in 0.001s
-----
Ran 4 tests in 0.001s
Ran 4 tests in 0.001s

OK
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

This code demonstrates **Test Driven Development (TDD)** using Python's unittest framework to test a `square(x)` function. The test cases are written before the function implementation to define the expected behavior. The tests verify the function's correctness for **positive numbers, zero, negative numbers, and decimal values**. Each test uses `assertEqual()` to compare the actual output with the expected result. The `unittest.main(verbosity=2)` statement runs all tests and provides detailed output. Once the `square(x)` function is implemented correctly in a separate file, all test cases pass, completing the TDD cycle of **test** → **fail** → **implement** → **pass**.

Task 2: Email Validation for a User Registration System

Prompt: Write Python unit tests using unittest for a `validate_email(email)` function following TDD. After tests fail, implement `validate_email(email)` in a separate file so all tests pass.

```

test_email.py > TestEmailValidation > test_invalid_format
1  """
2  Write Python unit tests using unittest for a validate_email(email) function following TDD.
3  After tests fail, implement validate_email(email) in a separate file so all tests pass.
4  """
5  import unittest
6  from email_test import validate_email
7  class TestEmailValidation(unittest.TestCase):
8      def test_valid_email(self):
9          result = validate_email("user@gmail.com")
10         print("Valid email test:", result)
11         self.assertTrue(result)
12     def test_missing_at(self):
13         result = validate_email("usergmail.com")
14         print("Missing @ test:", result)
15         self.assertFalse(result)
16     def test_missing_domain(self):
17         result = validate_email("user@")
18         print("Missing domain test:", result)
19         self.assertFalse(result)
20     def test_invalid_format(self):
21         result = validate_email("user@com")
22         print("Invalid format test:", result)
23         self.assertFalse(result)
24 if __name__ == "__main__":
25     unittest.main(verbosity=2)
26

```

```

def validate_email(email):
    return "@" in email and "." in email

```

Output:

```

17         result = validate_email("user@")
18         print("Missing domain test:", result)
19         self.assertFalse(result)
20     def test_invalid_format(self):
21         result = validate_email("user@com")
22         print("Invalid format test:", result)
23         self.assertFalse(result)
24 if __name__ == "__main__":
25     unittest.main(verbosity=2)
26

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> python test_email.py
test_invalid_format (__main__.TestEmailValidation.test_invalid_format) ... Invalid
ok
test_missing_at (__main__.TestEmailValidation.test_missing_at) ... Missing @ test: False
ok
test_missing_domain (__main__.TestEmailValidation.test_missing_domain) ... Missing domain test: False
ok
test_valid_email (__main__.TestEmailValidation.test_valid_email) ... Valid email test: True
ok

-----
Ran 4 tests in 0.001s

OK
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>

```

Explanation: This task applies **Test Driven Development (TDD)** to validate email addresses. Unit tests are written first using the unittest framework to check valid and invalid email formats such as missing @, missing domain, and incorrect structure. Initially, the tests fail

because the function is not implemented. After defining the expected behavior through tests, the `validate_email()` function is implemented in a separate file. Once implemented, all test cases pass, demonstrating correct email validation and proper application of the TDD cycle.

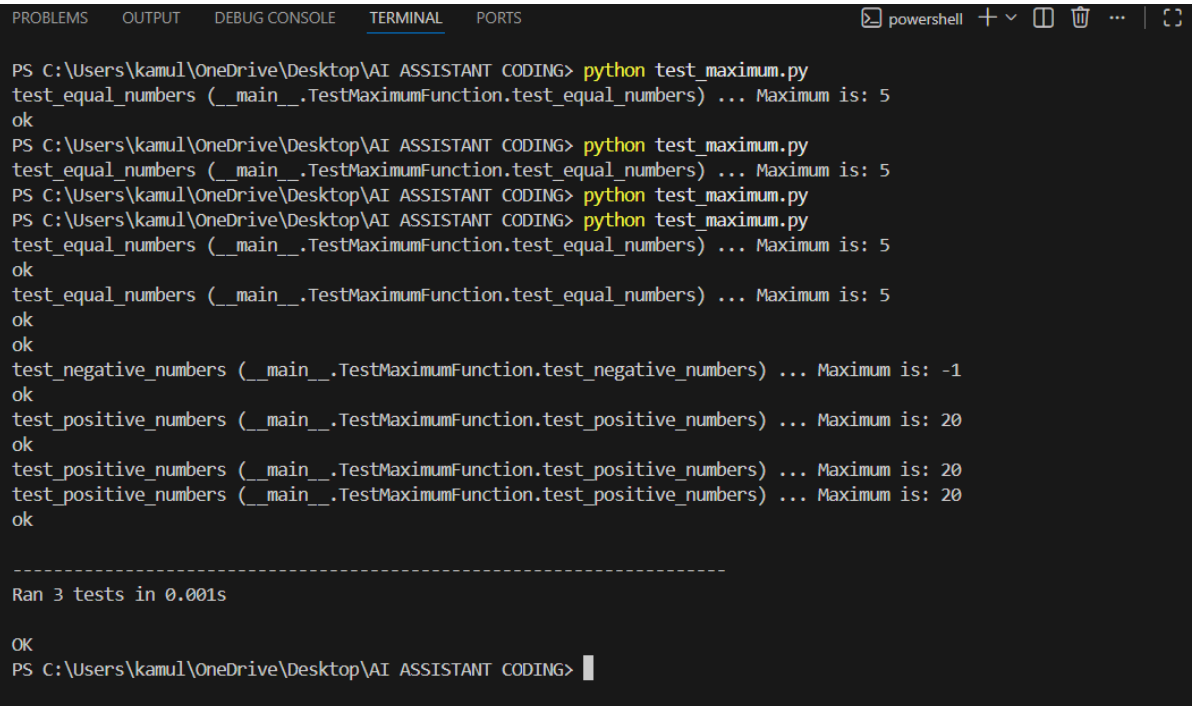
Task 3: Decision Logic Development Using TDD

Prompt: Write Python unit tests using unittest for a `find_max(a, b, c)` function following TDD. After tests fail, implement `find_max(a, b, c)` in a separate file so all tests pass.

```
test_maximum.py > ...
1  '''
2  Write Python unit tests using unittest for a find_max(a, b, c) function following TDD.
3  After tests fail, implement find_max(a, b, c) in a separate file so all tests pass.
4  '''
5  import unittest
6  from maximum_test import find_max
7
8  class TestMaximumFunction(unittest.TestCase):
9
10     def test_positive_numbers(self):
11         result = find_max(10, 20, 15)
12         print("Maximum is:", result)
13         self.assertEqual(result, 20)
14
15     def test_negative_numbers(self):
16         result = find_max(-1, -5, -3)
17         print("Maximum is:", result)
18         self.assertEqual(result, -1)
19
20     def test_equal_numbers(self):
21         result = find_max(5, 5, 5)
22         print("Maximum is:", result)
23         self.assertEqual(result, 5)
24
25 if __name__ == "__main__":
26     unittest.main(verbosity=2)
27
```

```
def find_max(a, b, c):  
    return max(a, b, c)
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... [ ]  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> python test_maximum.py  
test_equal_numbers (__main__.TestMaximumFunction.test_equal_numbers) ... Maximum is: 5  
ok  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> python test_maximum.py  
test_equal_numbers (__main__.TestMaximumFunction.test_equal_numbers) ... Maximum is: 5  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> python test_maximum.py  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> python test_maximum.py  
test_equal_numbers (__main__.TestMaximumFunction.test_equal_numbers) ... Maximum is: 5  
ok  
test_equal_numbers (__main__.TestMaximumFunction.test_equal_numbers) ... Maximum is: 5  
ok  
ok  
test_negative_numbers (__main__.TestMaximumFunction.test_negative_numbers) ... Maximum is: -1  
ok  
test_positive_numbers (__main__.TestMaximumFunction.test_positive_numbers) ... Maximum is: 20  
ok  
test_positive_numbers (__main__.TestMaximumFunction.test_positive_numbers) ... Maximum is: 20  
test_positive_numbers (__main__.TestMaximumFunction.test_positive_numbers) ... Maximum is: 20  
ok  
-----  
Ran 3 tests in 0.001s  
OK  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> |
```

Explanation: In this task, TDD is used to determine the maximum value among three inputs. Unit tests are created first to cover different scenarios including positive numbers, negative numbers, and equal values. These tests clearly define the expected output before any logic is written. After the tests fail initially, the `find_max()` function is implemented based strictly on the test requirements. All tests pass successfully, ensuring accurate decision logic and validating the effectiveness of test-driven development.

Task 4: Shopping Cart Development with AI-Assisted TDD

Prompt: Write Python unit tests using unittest for a `ShoppingCart` class following TDD. After tests fail, implement `ShoppingCart` in a separate file so all tests pass.

```

1  """
2  Write Python unit tests using unittest for a ShoppingCart class following TDD.
3  After tests fail, implement ShoppingCart in a separate file so all tests pass.
4  """
5  import unittest
6  from cart_test import ShoppingCart
7  class TestShoppingCart(unittest.TestCase):
8      def setUp(self):
9          self.cart = ShoppingCart()
10     def test_add_item(self):
11         self.cart.add_item(100)
12         print("Total after adding item:", self.cart.total_price())
13         self.assertEqual(self.cart.total_price(), 100)
14     def test_remove_item(self):
15         self.cart.add_item(100)
16         self.cart.remove_item(100)
17         print("Total after removing item:", self.cart.total_price())
18         self.assertEqual(self.cart.total_price(), 0)
19     def test_multiple_items(self):
20         self.cart.add_item(50)
21         self.cart.add_item(30)
22         print("Total price:", self.cart.total_price())
23         self.assertEqual(self.cart.total_price(), 80)
24 if __name__ == "__main__":
25     unittest.main(verbosity=2)

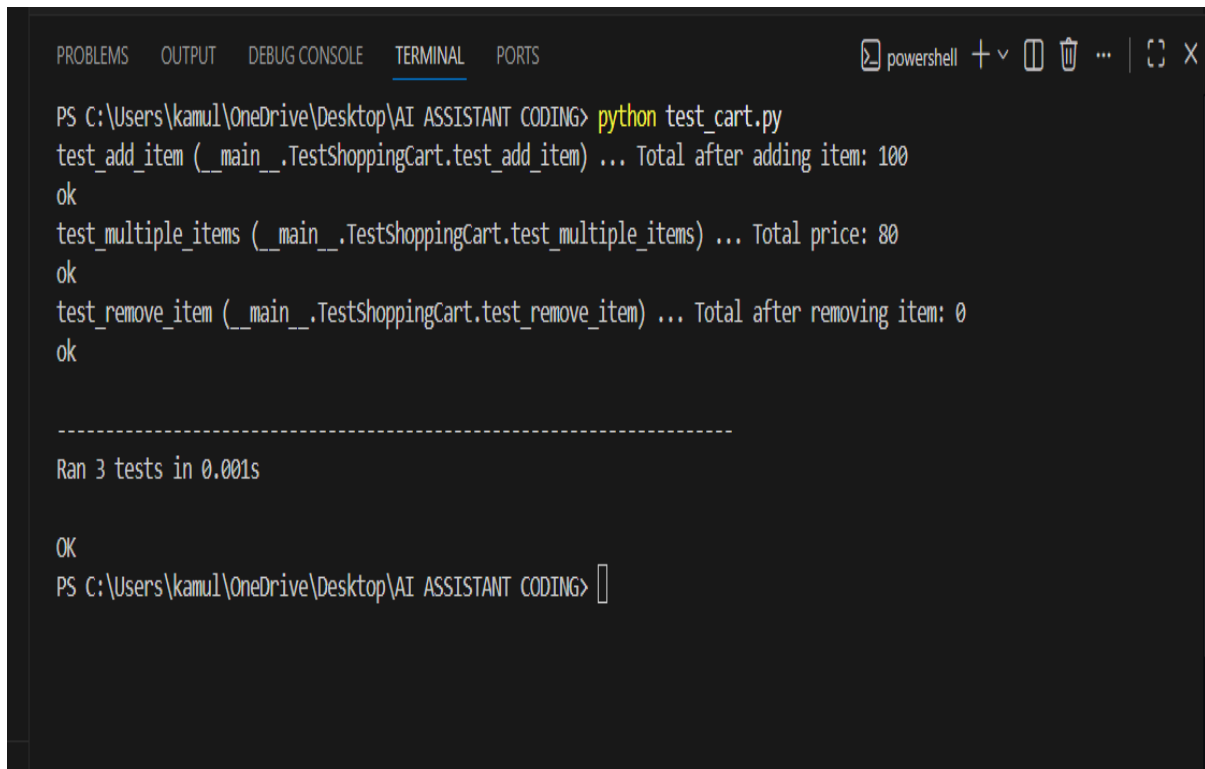
```

```

cart_test.py > ...
1  class ShoppingCart:
2
3      def __init__(self):
4          self.items = []
5
6      def add_item(self, price):
7          self.items.append(price)
8
9      def remove_item(self, price):
10         if price in self.items:
11             self.items.remove(price)
12
13     def total_price(self):
14         return sum(self.items)
15

```

Output:



```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> python test_cart.py
test_add_item (__main__.TestShoppingCart.test_add_item) ... Total after adding item: 100
ok
test_multiple_items (__main__.TestShoppingCart.test_multiple_items) ... Total price: 80
ok
test_remove_item (__main__.TestShoppingCart.test_remove_item) ... Total after removing item: 0
ok

-----
Ran 3 tests in 0.001s

OK
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: This task demonstrates TDD in a **class-based design**. Unit tests are written to verify core shopping cart behaviors such as adding items, removing items, and calculating the total price. The tests act as a specification for how the ShoppingCart class should behave. After test creation, the class and its methods are implemented to satisfy the test cases. Successful test execution confirms correct cart functionality and highlights the importance of behavior-driven testing in software development.

Task 5: String Validation Module Using TDD

Prompt: Write Python unit tests using unittest for an `is_palindrome(text)` function following TDD. After tests fail, implement `is_palindrome(text)` in a separate file so all tests pass.

```
test_palindrome.py > TestPalindromeFunction > test_case_insensitive
1 """
2 Write Python unit tests using unittest for an is_palindrome(text) function following TDD.
3 After tests fail, implement is_palindrome(text) in a separate file so all tests pass.
4 """
5 import unittest
6 from palindrome_test import is_palindrome
7 class TestPalindromeFunction(unittest.TestCase):
8     def test_simple_palindrome(self):
9         result = is_palindrome("madam")
10        print("Palindrome test:", result)
11        self.assertTrue(result)
12    def test_not_palindrome(self):
13        result = is_palindrome("hello")
14        print("Not palindrome test:", result)
15        self.assertFalse(result)
16    def test_case_insensitive(self):
17        result = is_palindrome("Madam")
18        print("Case insensitive test:", result)
19        self.assertTrue(result)
20 if __name__ == "__main__":
21     unittest.main(verbosity=2)
22
```

```
palindrome_test.py > ...
1 def is_palindrome(text):
2     text = text.lower()
3     return text == text[::-1]
4
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... [ ] [ ] [ ] [ ]
test_not_palindrome (__main__.TestPalindromeFunction.test_not_palindrome) ... Not palindrome test: False
ok
test_simple_palindrome (__main__.TestPalindromeFunction.test_simple_palindrome) ... Palindrome test: True
ok

-----
Ran 3 tests in 0.001s

OK
PS C:\Users\kamu1\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: This task uses TDD to build a reliable palindrome checking function. Unit tests are written first to handle simple palindromes, non-palindromes, and case-insensitive inputs. The tests initially fail, guiding the implementation of the `is_palindrome()` function. Once implemented, all tests pass, confirming correct string validation. This task demonstrates how TDD helps in handling edge cases and ensures robust text-processing logic.